

Q.No:10

Question description

Kaaleppi has just robbed a bank and is now heading to the harbor. However, the police wants to stop him by closing some streets of the city.

What is the minimum number of streets that should be closed so that there is no route between the bank and the harbor?

Constraints

- $2 \leq n \leq 500$
- $1 \leq m \leq 1000$
- $1 \leq a, b \leq n$

Input

The first input line has two integers n and m : the number of crossings and streets. The crossings are numbered $1, 2, \dots, n$. The bank is located at crossing 1, and the harbor is located at crossing n .

After this, there are m lines that describe the streets. Each line has two integers a and b : there is a street between crossings a and b . All streets are two-way streets, and there is at most one street between two crossings.

Output

First print an integer k : the minimum number of streets that should be closed. After this, print k lines describing the streets. You can print any valid solution.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4 5
1 2
1 3
2 3
3 4
1 4
```

EXPECTED OUTPUT

```
2
1 4
3 4
```

Test Case 2

INPUT (STDIN)

```
4 5
1 1
2 2
2 3
3 4
1 4
```

EXPECTED OUTPUT

```
1
1 4
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void link(int i,int h)
```

Test Case 2

KEYWORD

```
int bfs(int n,int s,int t)
```

Complexity Test Cases

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX 505
#define INF 1000000000

int capacity[MAX][MAX];
int parent[MAX];
int edgeA[2005];
int edgeB[2005];
int edgeCount = 0;
```

```
void link(int i, int j)
{
    capacity[i][j]++;
    capacity[j][i]++;

    edgeA[edgeCount] = i;
    edgeB[edgeCount] = j;
    edgeCount++;
}

int bfs(int n, int s, int t)
{
    int queue[MAX];
    int visited[MAX];

    int front = 0;
    int rear = 0;

    int i, u, v;

    memset(visited, 0, sizeof(visited));
    queue[rear++] = s;
    visited[s] = 1;
    parent[s] = -1;

    while (front < rear)
    {
        u = queue[front++];

        for (v = 1; v <= n; v++)
        {
            if (!visited[v] && capacity[u][v] > 0)
            {
                visited[v] = 1;
                parent[v] = u;

                queue[rear++] = v;

                if (v == t)
                    return 1;
            }
        }
    }

    return 0;
}

int main()
{
    int n, m;
    int i;
```

```
int a, b;

int maxFlow = 0;

scanf("%d %d", &n, &m);

memset(capacity, 0, sizeof(capacity));

for (i = 0; i < m; i++)
{
    scanf("%d %d", &a, &b);

    link(a, b);
}
while (bfs(n, 1, n))
{
    int v = n;

    while (v != 1)
    {
        int u = parent[v];

        capacity[u][v]--;
        capacity[v][u]++;

        v = u;
    }

    maxFlow++;
}
int reachable[MAX];
int queue[MAX];

int front = 0;
int rear = 0;

memset(reachable, 0, sizeof(reachable));

queue[rear++] = 1;
reachable[1] = 1;

while (front < rear)
{
    int u = queue[front++];

    for (i = 1; i <= n; i++)
    {
        if (!reachable[i] && capacity[u][i] > 0)
        {
            reachable[i] = 1;
            queue[rear++] = i;
        }
    }
}
```

```
}  
}  
  
printf("Output:\n");  
printf("%d\n", maxFlow);  
  
for (i = 0; i < edgeCount; i++)  
{  
    a = edgeA[i];  
    b = edgeB[i];  
  
    if (reachable[a] != reachable[b])  
    {  
        printf("%d %d\n", a, b);  
    }  
}  
  
return 0;  
}
```

Output:

Test_case:1

```
adityajaiswal/Desktop  
4 5  
1 2  
1 3  
2 3  
3 4  
1 4  
Output:  
2  
3 4  
1 4  
aadityajaiswal@Aad
```

Test_case: 2

```
adityajaiswal/Desktop  
4 5  
1 1  
2 2  
2 3  
3 4  
1 4  
Output:  
1  
1 4  
aadityajaiswal@Aad
```